

IT342 – Phase 2: Mobile Development

PayWeac | Carin | IT342

March 28, 2026

1. Submission Details

Student	Carin
App Name	PayWeac
Repository	https://github.com/fredjohn909-blip/IT342-Carin-Payweac
Commit Msg	IT342 Phase 2 – Mobile Development Completed
Mobile Path	mobile/ (inside same GitHub repository)
Tech Stack	Kotlin, XML Layouts, Retrofit 2, Coroutines, Material Components

2. How Registration Works

Screen & Input Fields

- The **RegisterActivity** presents three fields: **Full Name**, **Email Address**, and **Password**.
- Fields are built with **Material TextInputLayout** (OutlinedBox style) for clear error feedback.
- A **ProgressBar** appears during the network call; the Register button is disabled to prevent duplicate submissions.

Input Validation (client-side)

- Name, Email, and Password fields must not be empty.
- Email is checked against **android.util.Patterns.EMAIL_ADDRESS**.
- Password must be at least **6 characters**.
- The first failing field receives focus and shows an inline error message.

API Call

On valid input, a **POST** request is sent to **/api/auth/register** carrying a JSON body with the user's name, email, and password. Retrofit serialises the **RegisterRequest** data class via Gson. The call runs inside a **lifecycleScope coroutine** so it never blocks the UI thread.

```
POST /api/auth/register
Content-Type: application/json

{
```

```
"name": "Juan Dela Cruz",  
"email": "juan@example.com",  
"password": "secret123"  
}
```

Success / Error Handling

- HTTP 2xx + **success: true** → Toast "Registration successful!" → navigate to LoginActivity.
- Backend error message → Toast with the server's message string.
- Network exception (no connection, timeout) → Toast with the exception message.

3. How Login Works

Screen & Input Fields

- The **LoginActivity** presents **Email Address** and **Password** fields.
- Same Material OutlinedBox style and ProgressBar/button-disable pattern as registration.

API Call

A **POST** request is sent to **/api/auth/login** with a **LoginRequest** JSON body. The backend validates credentials and returns a JWT token on success.

```
POST /api/auth/login  
Content-Type: application/json  
  
{  
  "email": "juan@example.com",  
  "password": "secret123"  
}  
  
// Success Response  
{ "success": true, "token": "", "message": "Login successful" }  
  
// Failure Response  
{ "success": false, "message": "Invalid email or password." }
```

Token Storage & Navigation

- On success, the JWT token is saved to **SharedPreferences** under key **auth_token**.
- The user is redirected to **DashboardActivity** (the post-login screen).
- The Dashboard displays a welcome message and a Logout button that clears the token.
- Invalid credentials → Toast with the server error message; user stays on Login screen.

4. API Integration

Libraries Used

Library	Version	Purpose
Retrofit 2	2.9.0	HTTP client — sends requests to the Phase 1 backend
Gson Converter	2.9.0	Converts Kotlin data classes to/from JSON automatically
Kotlinx Coroutines	1.7.3	Non-blocking async network calls on IO dispatcher
Lifecycle Runtime KTX	2.7.0	lifecycleScope for coroutine lifecycle management
Material Components	1.11.0	TextInputLayout, buttons, and theming

Key Architecture Decisions

- **ApiClient** (singleton object) — creates one Retrofit instance reused across the app.
- **ApiService** interface — declares **suspend** functions so they work directly with coroutines.
- **usesCleartextTraffic="true"** in AndroidManifest allows plain HTTP during development/testing.
- Base URL uses **10.0.2.2** (Android emulator alias for host machine localhost) — update to your server IP for physical device testing.

5. Mobile Project File Structure

```
mobile/
├── app/
├── build.gradle
├── src/main/
│   ├── AndroidManifest.xml
│   ├── java/com/carin/payweac/
│   │   ├── api/
│   │   │   ├── ApiClient.kt # Retrofit singleton
│   │   │   ├── ApiService.kt # API endpoint definitions
│   │   │   └── model/
│   │   │       ├── Models.kt # Request/Response data classes
│   │   │       └── ui/
│   │   │           ├── register/
│   │   │           │   ├── RegisterActivity.kt
│   │   │           │   └── login/
│   │   │           │       ├── LoginActivity.kt
│   │   │           │       └── dashboard/
│   │   │           │           ├── DashboardActivity.kt
│   │   │           └── res/layout/
│   │   │               ├── activity_register.xml
│   │   │               ├── activity_login.xml
│   │   │               └── activity_dashboard.xml
```

6. Screenshots

The screenshots below should be replaced with actual screenshots from your running app. Attach them in this section before final submission.

#	Screenshot	Description
1	Registration Screen	Shows Name, Email, Password fields and Register button
2	Successful Registration	Toast message confirming registration success
3	Login Screen	Shows Email, Password fields and Log In button
4	Successful Login	Toast message confirming login success
5	After Login Screen	Dashboard / home screen displayed after login
6	Database Record	Backend database showing newly registered user record

Note: Run the app on an emulator or physical device connected to your Phase 1 backend, then capture screenshots for each of the 6 items above and insert them into this document before submission.